- **Wi-Fi drivers:** The Galileo supports Intel Wi-Fi cards via drivers included in the Linux image. There are many mini-PCIe Wi-Fi cards.
- **Python:** There are Python scripts readily available that can check for unread emails or perform many other small tasks easily. Custom Python scripts can also be easily built to enhance functionality and truly make your device, a device connected to the Internet of Things.
- **OpenCV:** OpenCV is an open-source computer vision library. Using a webcam connected to the USB port of the Galileo board a live feed from the camera can be captured to perform tasks such as object detection or recognition.
- **SSH:** Secure Shell (SSH) is a command line tool/protocol to securely access a remote computer. This can enable applications that, say, remote control a Galileo that is monitoring a home or allow plain communication with the Galileo board without the serial communication port.
- **Node.js:** Node.js is a library dedicated to building server-side applications in Javascript. Node.js is meant to run on HTTP server and its applications are event driven. Most suited for web projects.
- **ALSA:** Advanced Linux Sound Architecture (ALSA) enables the Galileo board to play sound.
- **V4L2:** Video4Linux2 is a video recording and playing Linux utility.

INSTALLING LINUX

To use Linux on the Galileo board, you will need at least a 1GB SD card. The image file (LINUX IMAGE for SD for Intel Galileo) can be downloaded from http://dgit.in/intelcommunitiesdoc22226. Windows users can use 7-Zip to extract the downloaded file, while Mac users can use The Unarchiver for extraction.

1. Extract the contents of the .7z file to the SD card (without putting it in a folder).
2. Turn off the Galileo board
3. Plug in the SD card
4. Switch on the Galileo board

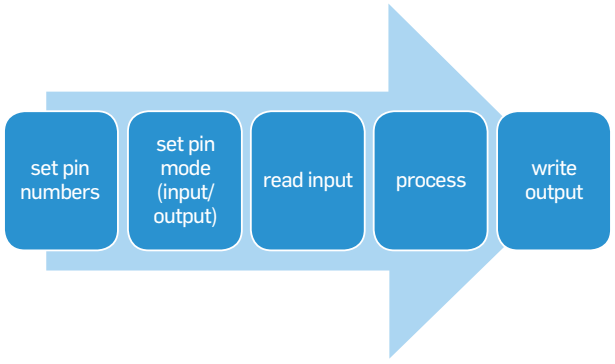
The first boot takes some time to set up SSH keys.

THE GALILEO COMMUNITY

Intel will be donating 50,000 Arduino-compatible development boards featuring Intel architecture to 1,000 universities around the world. Since the Intel Galileo University Donation

```
char output[3];

void setup() {

  Serial.begin(115200);

  system("echo '#!/usr/bin/python' > myScript.py");
  system("echo 'import time' >> myScript.py");
  system("echo 'for i in range(10):' >> myScript.py");
  system("echo '    with open(\"log.txt\", \"w\") as fh:' >>
myScript.py");
  system("echo '        fh.write(\"{0}\".format(i))' >> myScript.
py");
  system("echo '    time.sleep(1)' >> myScript.py");
  system("chmod a+x myScript.py");
  system("./myScript.py &");
}

void loop() {

  FILE *fp;
  fp = fopen("log.txt", "r");
  fgets(output, 2, fp);
  fclose(fp);
  Serial.println(output);
  delay(1000);

}
```

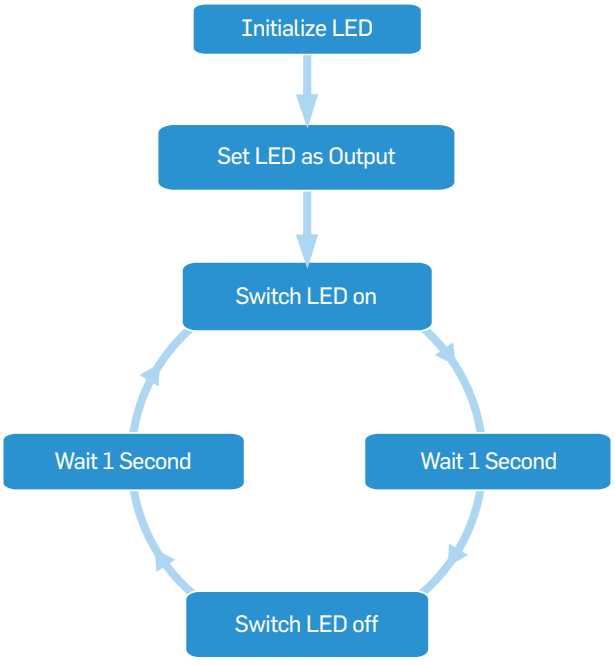
Arduino code for executing Linux code on Galileo board



Code flow for hardware programming

```
/*
  Blink
  Turns on an LED on for one second, then off for one second, repeatedly.

  This example code is in the public domain.
 */

int led = 13;

void setup() {
  pinMode(led, OUTPUT);
}

void loop() {
  digitalWrite(led, HIGH);   // turn on the LED
  delay(1000);               // wait for a second
  digitalWrite(led, LOW);    // turn off the LED
  delay(1000);               // wait for a second
}
```

Arduino code for Blinking of LED at Pin 13



Flowchart for Blinking of LED at Pin 13